Univerzitet u Nišu
Elektronski fakultet

Tinder Clone

Predmet:
Napredne baze podataka
2026.

Studenti:
David Marković
Jovan Bogdanović

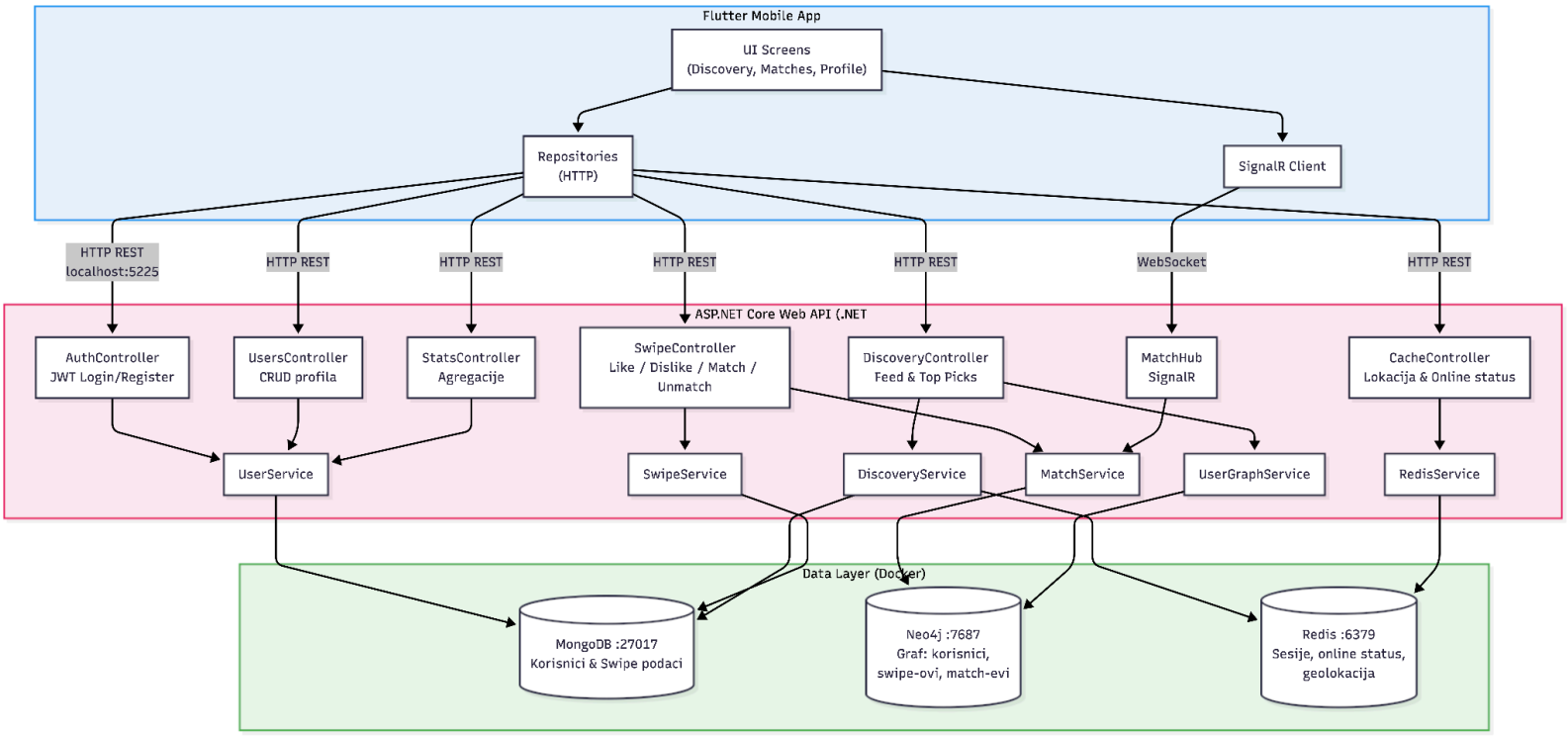
2026.

Sadržaj

Arhitektura sistema....2
Baze podataka i njihove uloge....2
Komunikacioni tokovi....2
Realizacija frontenda....3
Struktura....3
Ključne tehnologije....3
Ekrani....3
Realizacija backenda....4
MongoDB – dokument baza korisnika....4
Neo4j – graf baza za relacije....4
Model grafa....5
Način rada aplikacije....5
Tok registracije....5
Tok discovery feed-a....5
Zaključak....6

Arhitektura sistema

Sistem je organizovan u tri odvojena sloja koji komuniciraju na jasno definisan način.



Baze podataka i njihove uloge

- MongoDB: Primarni izvor podataka – profili korisnika, lozinke, preferencije, lokacija
- Neo4J: Relacioni podaci – ko je koga lajkovao, match-evi, zajednička interesovanja
- Redis: Performansni sloj – geolokacija, online status, match notifikacije

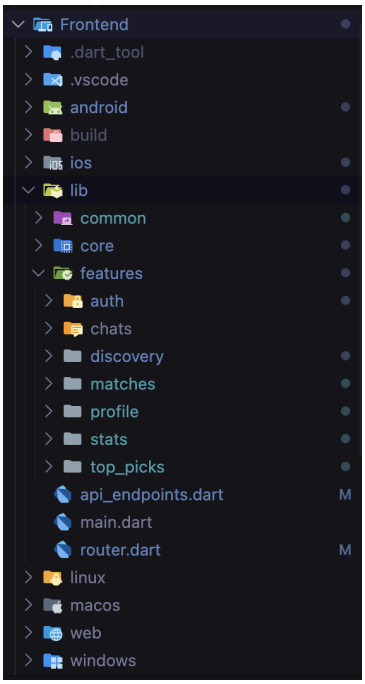
Komunikacioni tokovi

- Flutter → Backend: HTTP REST pozivi za sve CRUD operacije
- Backend → Flutter (real-time): WebSocket konekcija putem SignalR - server šalje match notifikaciju klijentu bez da klijent mora da pita.
- Backend → MongoDB: Svaki zahtev koji čita ili mijenja profil korisnika, swipe zapis ili lokaciju.
- Backend → Neo4j: Svaki zahtev vezan za lajkove, dislike-ove, match-eve i preporuke korisnika.
- Backend → Redis: Svaki zahtev za online status, geolokaciju, proximity pretragu

Realizacija frontenda

Frontend je razvijen kao cross-platform mobilna aplikacija koristeći Flutter framework sa Dart programskim jezikom.

Struktura



api_endpoints.dart ← Centralizovane URL konstante
router.dart ← GoRouter konfiguracija (sve rute)
common ← Dijeljeni modeli (UserModel, UserPreferences...)
core ← Shared servisi (SignalR, CacheRepository)
features/
- **auth** ← Login, registracija, kreiranje profila
- **discovery** ← Swipe feed
- **matches** ← Lista match-eva, pregled profila
- **profile** ← Profil + edit profil
- **top_picks** ← Top korisnici po broju lajkova
- **stats** ← Statistike aplikacije

Ključne tehnologije

- **go_router** – Deklarativni routing sa extra parametrima za prosljeđivanje objekata između ekrana
- **shared_preferences** – Lokalno čuvanje ID-a korisnika
- **signalr_netcore** – SignalR klijent za real-time match notifikacije
- **geolocator** –Lociranje GPS lokacije uređaja
- **http** – HTTP klijent za REST pozive

Ekrani

- **DiscoveryScreen** - Kartice korisnika sa swipe gestom (levo = dislike, desno = like).
- **MatchesScreen** - Grid prikaz svih match-eva sa online indikatorom. Real-time se ažurira putem SignalR.
- **UserProfileViewScreen** - Detaljan profil match-a sa mogućnošću unmatch-a

- **ProfileScreen** - Pregled vlastitog profila; linkovi ka editovanju, statistikama i logu out
- **EditProfileScreen** - Forma za izmenu svih polja profila (ime, godine, bio, pol, preferencije, interesovanja)
- **TopPicksScreen** - Korisnici sa najviše lajkova na platformi
- **StatsScreen** - Statistike iz MongoDB (Aggregation Pipeline)

Realizacija backenda

Backend je razvijen kao ASP.NET Core Web API koristeći .NET 9 i organizovan po principu: Controller → Service → Repository.

MongoDB – dokument baza korisnika

MongoDB čuva sve korisničke podatke. Model User je mapiran direktno na MongoDB kolekciju pomoću atributa:

```
public class User
{
   [BsonId]
   [BsonRepresentation(BsonType.ObjectId)]
   public string Id { get; set; }
   public string Email { get; set; }
   public string PasswordHash { get; set; }
   public string FirstName { get; set; }
   public string LastName { get; set; }
   public int Age { get; set; }
   public string Bio { get; set; }
   public UserPreferences UserPreferences { get; set; }
   public GenderEnum Gender { get; set; }
   public List<string>? Interests { get; set; }
   public UserLocation? LastLocation { get; set; }
}
```

Lokacija se čuva u GeoJSON formatu ([longitude, latitude]) što MongoDB-u omogućava geospacijalne upite direktno na kolekciji. (Ovo sluzi kao FallBack u slucaju da se Redis ugasio i izgubio prethodne podatke, Mongo uzima poslednje zapamcene kordinate i salje ih u Redis)

Neo4j – graf baza za relacije

Neo4j čuva relacionu strukturu između korisnika u obliku čvorova i veza. Ovo je centralna baza za sve swipe i match operacije.

Model grafa

(:User)-[:LIKES]->(:User)
(:User)-[:DISLIKES]->(:User)
(:User)-[:BLOCKED]->(:User)
(:User)-[:INTERESTED_IN]->(:Interest)

LikeUser – detekcija match-a
Kada korisnik lajkuje drugu osobu, Neo4j jednim upitom upisuje vezu i provjerava postoji li uzajamni like.
Graf agregacija broji sve dolazne LIKES veze za svakog korisnika i sortira po popularnosti.
Redis – in-memory keš
Redis se koristi kao performansni sloj za tri kategorije podataka:
1. Geolokacija – GeoAdd i GeoRadius
2. Online status – TTL ključevi
3. Match notifikacije – kratkotrajna upozorenja

Način rada aplikacije

Tok registracije

1. Korisnik unosi email i lozinku → backend hešuje lozinku sa BCrypt
2. Korisnik bira pol i preferencije (OrientationScreen)
3. Korisnik bira interesovanja iz fiksnog seta (InterestsScreen)
4. Podaci se čuvaju u MongoDB, interesovanja se upisuju i u Neo4j kao [:INTERESTED_IN] veze

Tok discovery feed-a

Svaki put kada korisnik otvori discovery tab:
1. Aplikacija šalje svoju GPS lokaciju na server
2. Server upisuje koordinate u Redis (GeoAdd)
3. Server dohvata ID-eve korisnika u krugu od 50km iz Redis-a (GeoRadius)
4. MongoDB vraća pune profile tih korisnika
5. Profili se filtriraju po preferencijama (pol, starosni raspon)
6. Neo4j sortira preostale korisnike po zajedničkim interesovanjima i isključuje već svajpovane
7. Lista stiže na Flutter, kartica sa osobom sa najviše poklapanja nalazi se na vrhu

GetDiscoveryFeed – centralna funkcija sistema

GetDiscoveryFeed je najkompleksnija funkcija i demonstrira kako se sve tri baze koriste zajedno u jednom toku:

Korak 1: Redis → GetUserLocationAsync(userId)
Ako nema u Redisu → MongoDB fallback → upiši u Redis
Korak 2: MongoDB → GetUserByIdAsync(userId)
(dohvati preferencije: pol, min/max godine)
Korak 3: Redis → GetNearbyUserIdsAsync(userId, 50km)
(GeoRadius pretraga unutar 50 kilometara)
Ako Redis nema lokacije → MongoDB → GetUsersByPreferencesAsync
Korak 4: MongoDB → GetUsersByIdsAsync(nearbyIds)
(dohvati pune profile bliskih korisnika)
Korak 5: Filtriranje u memoriji
- pol mora odgovarati preferencama tražioca
- godine moraju biti u dozvoljenom rasponu
Korak 6: Neo4j → GetRecommendations(userId, filteredIds)
- isključuje već svajpovane korisnike
- sortira po broju zajedničkih interesovanja
Korak 7: MongoDB → GetUsersByIdsAsync(recommendedIds)
- dohvati finalne profile sortirane po Neo4j skoru

Zaključak

Projekat demonstrira kako tri različite baze podataka mogu biti integrisane u jedinstven sistem, pri čemu svaka doprinosi u segmentu za koji je optimizovana:

- **MongoDB** - pruža fleksibilnost dokumentnog modela za heterogene korisničke profile i moćne analitičke mogućnosti kroz Aggregation Pipeline
- **Neo4j** - rešava problem koji relacione baze rešavaju neefikasno, detekcija uzajamnih lajkova i rangiranje korisnika po zajedničkim osobinama u jednom Cypher upitu.
- **Redis** - geolokacijska pretraga u realnom vremenu putem Geohash algoritma, session management putem TTL ključeva i instant notifikacije koje prethode WebSocket poruci.

Kombinacija ove tri paradigme – dokumentna, graf i in-memory – pokazuje da savremene aplikacije retko mogu biti optimalno implementirane uz jednu univerzalnu bazu podataka, već zahtejevaju odabir alata prema prirodi podataka i načinu njihovog korišćenja.